

— PRODUCTION ARCHITECTURE REVIEW

Vivaha Matrimony Platform

Reverse Flow Calculation & Backend Module Mapping

- Customer
- Moderation
- Admin
- Support
- Payments
- Trust & Safety

Archive → Registration / 29 lifecycle stages traced backward through backend/internal and frontend/lib/features

— DOCUMENT PURPOSE

A production-oriented review, read backward from account closure to first OTP.

Vivaha is a Go + Flutter matrimony platform: profile creation, matchmaking, chat and calling, premium unlocks, and moderation. This review walks the lifecycle in reverse — from an archived account back to registration — so every stage is tied to the backend package that actually owns it.

1

Reverse Walkthrough

Every stage from Archive back to Registration — what happens and why, in the order an account's data actually resolves.

2

Module Mapping

Each stage tied to the exact `internal/` package in the Go backend and the `features/` folder in the Flutter app.

3

Cross-Cutting Systems

Middleware, moderation, notifications and cache — the modules that run underneath every stage, not tied to one step.

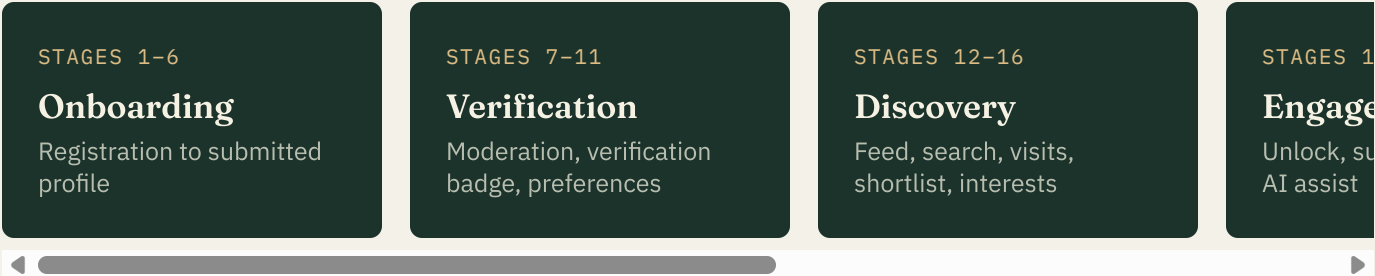
4

System Architecture

How the Customer, Admin and background-worker binaries connect through the API layer to data and integrations.

— THE FORWARD FLOW, AT A GLANCE

29 stages, grouped into 6 phases — the spine this review runs backward across.



WHY CALCULATE IT IN REVERSE?

Matching the credit-review method to a matchmaking platform.

FORWARD VIEW

Builds the member journey

- Shows how a member moves from OTP login to a live conversation
- Good for onboarding new team members to the product
- Answers: "what happens next?"

REVERSE VIEW

Validates the system that runs it

- Starts from an archived account and asks what had to be true before it
- Surfaces every dependency: no archive without deactivation, no chat without an unlock decision
- Answers: "what backend module makes this real?"

STATUS KEY – WHAT'S SETTLED VS IN MOTION

Every stage on the following pages is tagged with its current build signal.

✓ PACKAGE PRESENT

The owning `internal/` package exists in the backend and a matching Flutter feature folder is wired into the app shell.

🔴 ACTIVELY EVOLVING

Under live iteration right now — tracked by uncommitted changes and recent commits touching this stage's code.

EVIDENCE FOR "ACTIVELY EVOLVING" — CALLS

Working tree shows uncommitted edits across `internal/calls/repository.go`, `internal/calls/service.go`, `call_controller.dart`, `active_call_screen.dart` and `incoming_call_screen.dart`. Most recent commit: *"Add a deterministic backstop for a call remaining 'connected' after the other side hangs up."* This is the one lifecycle stage this review can date precisely — every other status below reflects package presence in the current tree, not a verified runtime audit.

STAGES 1–6

Onboarding

Where every member's journey actually begins — Registration to a submitted profile.

6

Review & Submit for Verification

✓ present

The onboarding wizard's final screen — `review_confirm_screen.dart` — locks the assembled profile and hands it to moderation before it can appear in discovery.

`internal/profiles, moderation`

Onboarding wizard → PostgreSQL profile record
→ moderation queue

5

Lifestyle, Diet & Photos

✓ present

Hobbies, marital status, height and diet are captured, and photos upload to object storage — the last data-heavy screens before submission.

`internal/profiles, storage`

`hobbies_screen, marital_height_diet_screen, photo_upload_screen` → Object Store

4

Education, Work & Family

✓ present

Qualification, work details and family background are captured as structured onboarding steps that later feed partner-preference matching.

internal/profiles

qualification_screen, work_details_screen, family_details_screen → PostgreSQL

3

Location, Religion & Community

✓ present

Location and religion/community fields anchor the profile in the dimensions matchmaking will later score against.

internal/profiles, reference

location_screen, religion_community_screen → internal/reference lookups

2

Profile For & Basic Details

✓ present

Who the profile is for (self or a relative), name and date of birth — the minimal identity shell every later screen builds on.

internal/users, profiles

profile_for_screen, name_dob_screen → internal/users, profiles

1

Registration & OTP Login

🔴 in motion

Mobile number entry and OTP verification — the device-bound identity anchor before any onboarding screen is reachable.

internal/auth, sms, devices

authentication/splash_screen → internal/auth, sms, devices

The splash/auth screen is among the frontend files currently modified in the working tree — this stage is under active touch, not confirmed stable.

STAGES 7–11

Verification

Turning a submitted profile into a discoverable, trusted one.

11

Recommendation & Search Index

Once verified, the profile is embedded into the ranking and search indices that `discover_matches` and `search` read from.

`internal/recommendation, search`

✓ present

`internal/recommendation, search` – index write on `verification.approved`

10

Matchmaking Score Computed

The scoring engine runs the new profile against the preference graph of existing members, producing the compatibility weights discovery will later read.

`internal/matchmaking`

✓ present

`internal/matchmaking/score.go` → PostgreSQL score table

9

Partner Preferences Captured

Age, height, community, location and other filters the member wants in a match — set once onboarding ends, editable from Settings.

`internal/preferences`

✓ present

`partner_preferences_screen` → `internal/preferences`

8

Identity / Photo Verification Badge

Once moderation clears the profile, a verified badge is issued — the trust signal shown across discovery and chat.

`internal/verification`

✓ present

Event → `internal/verification` → badge flag on profile

7

Profile Moderation Review

Photos and free-text fields are screened before a profile can surface in discovery; Admin can approve

`internal/moderation, admin`

✓ present

or send it back for changes.

internal/moderation → admin review queue

STAGES 12-16

Discovery

How one verified profile finds another.

16

Send Interest

A member sends interest to a discovered profile; the request is stored and a notification is queued for the recipient.

interests screen → internal/interests → internal/notifications

✓ present

internal/interests

15

Shortlist / Favourite

Profiles worth a second look are bookmarked for later, separate from the higher-commitment interest action.

favourites screen → internal/shortlisted, favourites

✓ present

internal/shortlisted, favourites

14

Profile Visit Logged

Viewing a profile is recorded so the "who viewed me" feed can surface it, gated by the visited member's privacy tier.

profile_visitors screen → internal/visitors

✓ present

internal/visitors

13

Search & Saved Searches

Members run filtered searches beyond the ranked feed, and can save a search to be alerted when new

✓ present

internal/search, savedsearches

matches fit it.
search screen → internal/search, savedsearches

12 Discover Matches Feed

The ranked, swipeable feed of candidate profiles — the primary discovery surface, backed by the matchmaking score and recommendation index.

discover_matches, demo_swipe → internal/matchmaking, recommendation

✓ present

internal/matchmaking, recommendation

STAGES 17–21 Engagement

From a mutual interest to an open conversation.

21 AI Chat Assist

Smart-reply and icebreaker suggestions surface inside the chat composer, backed by the dedicated AI package.

ai_chat feature → internal/ai

✓ present

internal/ai

20 Chat Opened

A message thread is created between the two members; every message is screened by chatguard for scam and abuse patterns before delivery.

chat feature → internal/chat, chatguard

✓ present

internal/chat, chatguard

19 Subscription / Coupon Applied

A premium plan purchase — optionally discounted by a coupon — raises the member's unlock tier.

✓ present

internal/subscriptions, payments, coupons

premium feature → internal/subscriptions,
payments, coupons

18 Unlock Gate

✓ present

Contact details and full chat access sit behind a
paywall check run before the member can proceed
past a mutual interest.

internal/unlock

unlock_gate screen → internal/unlock, payments

17 Interest Accepted / Declined

✓ present

The recipient acts on a sent interest; acceptance fires
the event that opens the unlock decision for both
members.

internal/interests,
notifications

internal/interests → Event →
internal/notifications

STAGES 22–26 Live Connection & Trust

Voice, video, and the guardrails that keep the platform safe.

26 Notifications & Digest

✓ present

Push, SMS and email nudges fire off platform events;
the scheduler binary runs periodic digest and
reminder jobs.

internal/notifications,
scheduler

internal/notifications, scheduler, email, sms
→ cmd/scheduler, cmd/notification

25 Support Ticket

✓ present

A grievance raised by a member is routed into a
support queue independent of any single
conversation's state.

internal/support

settings → internal/support

24

Report / Block

✓ present

An abusive or unwanted profile is reported or blocked, immediately removing it from that member's discovery and chat surfaces.

internal/reports, blocked

blocked_users feature → internal/reports, blocked

23

Call Reliability Backstop

🔴 in motion

A deterministic check now catches a call that should have ended but is still marked "connected" after the other side hangs up — closing a real gap in the signaling state machine.

internal/calls

internal/calls/service.go → call.ended reconciliation

Most recent commit on this codebase: "Add a deterministic backstop for a call remaining 'connected' after the other side hangs up."

22

Voice / Video Call

🔴 in motion

Call signaling — ringing, incoming, active, ended — runs over the websocket connection between the two members' apps.

internal/calls, websocket

active_call_screen, incoming_call_screen → internal/calls, websocket

Repository, service and both call screens are currently modified in the working tree — this stage is mid-iteration.

STAGES 27-29

Closure & Retention

Working backward from where a member's data finally rests.

29

Analytics Snapshot

Before archival, the account's lifecycle events are rolled into cohort and funnel reporting — the last read of the data before it's put away.

internal/analytics → reporting snapshot

✓ present

internal/analytics

28

Data Archive / Deletion

On permanent deletion, PII is purged and media is removed from object storage per retention policy; the cmd/worker binary handles the batch pass.

internal/users, storage → cmd/worker

✓ present

internal/users, storage

27

Account Deactivation

The profile is hidden from discovery and search but the record is retained — reversible, distinct from permanent deletion below it.

settings → internal/users → recommendation/search index removal

✓ present

internal/users

— END-TO-END ACTOR FLOW

The same 29 stages, seen through who actually touches each one.

1

Member Onboards

OTP login, wizard steps,
submits profile for review

2

Moderation Verifies

Photo/text screening, badge
issued, preferences and scoring
set up

3

Member Discovers

Feed, search, visits, shortlists,
sends interest

4

Members Connect

Unlock, chat, AI assist, voice/video call

Loop closes at Stage 21 (Chat Opened) — members then cycle between chat and calls indefinitely, with Trust & Safety and Support re-engaging only on a report, ticket, or eventual deactivation.

CROSS-CUTTING MODULES

**Running underneath every stage above
— not tied to a single step in the flow.**

`internal/middleware`

Middleware

Auth, rate limiting and RBAC checks on every API call before it reaches a domain package.

`internal/admin`

Admin

Backoffice access for moderation, support and reporting panels.

`internal/cache`

Cache

Redis-backed reads for discovery feeds, search and hot profile lookups.

internal/queue

Queue

Background job dispatch — the async spine between webhook receipt and downstream state change.

internal/logger

Logger

Structured logging across every binary — api, worker, scheduler, notification.

internal/devices

Devices

Device binding and push-token registration underneath auth and notifications.

internal/storage

Storage

Object storage for photos, verification documents and call artifacts.

email, sms

Delivery Channels

Outbound OTP, transactional and digest messages routed through provider integrations.

BACKEND ARCHITECTURE FLOW

Five binaries share one internal/ tree;
async workers keep member-facing APIs fast.

Customer App

Admin Panel



RECOMMENDED NEXT STEPS

Where this mapping earns its keep.

- 1

Validate the reverse map against the live codebase

This review infers stage behavior from package and screen names — walk each stage's handlers to confirm the mapping holds, not just that the package exists.
- 2

Close out the Calls signaling work

Repository, service and both call screens are mid-edit right now — land and verify the connected-state backstop before treating Stage 22–23 as settled.
- 3

Extend the same mapping to Admin and Support workflows

Carry this reverse-flow method into moderation queues and the grievance pipeline next.

4

Keep the architecture diagram as the shared reference

One picture for mobile, backend and admin teams to align around the same five-binary contract.

— THANK YOU

Vivaha — Reverse Flow & Backend Architecture Review

Prepared from the live `backend/internal` and `frontend/lib/features` trees as a companion to a full stage-by-stage technical mapping.

Module presence confirmed by directory listing; stage descriptions are architectural inference from naming and structure, not a line-by-line code audit — treat "✓ present" as "package exists," not "verified bug-free."